

ATP Tennis ML Project — Data Cleaning → Dataset Split

For Teammate: Complete everything up to the train/test split. Model training starts after this.

Target handoff

The teammate should give you a single prepared dataset containing the engineered pre-match features and a target column, already split chronologically into training (2018–2023) and testing (2024). You will start from model training.

1. Load and combine the 7 ATP CSV files

```
import pandas as pd
import glob
import os

files = sorted(glob.glob("atp_matches_*.csv"))

print("Files found:")
for f in files:
    print(" -", f)

if len(files) != 7:
    print("WARNING: Expected 7 CSV files (2018-2024).")

dfs = []

for f in files:
    df = pd.read_csv(f)

    year = int(
        os.path.basename(f).split("_")[-1].replace(".csv", "")
    )

    df["year"] = year
    dfs.append(df)

data = pd.concat(dfs, ignore_index=True)

print("Combined shape:", data.shape)
print(data.head())
```

2. Clean the dataset

```
# Remove exact duplicate rows
data = data.drop_duplicates().reset_index(drop=True)

# Standardize column names
data.columns = (
    data.columns
    .str.strip()
    .str.lower()
)

# Convert important numeric columns
numeric_cols = [
    "winner_rank", "loser_rank",
    "winner_rank_points", "loser_rank_points",
    "winner_age", "loser_age"
]

for col in numeric_cols:
    if col in data.columns:
        data[col] = pd.to_numeric(
            data[col], errors="coerce"
        )

# Clean categorical columns
categorical_cols = [
    "surface", "round", "tourney_name",
    "winner_name", "loser_name"
]

for col in categorical_cols:
    if col in data.columns:
        data[col] = data[col].astype("string").str.strip()

# Essential fields needed later
required_cols = [
```

```

    "winner_id", "loser_id",
    "winner_rank", "loser_rank",
    "winner_rank_points", "loser_rank_points",
    "surface"
]

data = data.dropna(
    subset=required_cols
).reset_index(drop=True)

# Age is not essential enough to delete matches.
# Impute missing ages with the median.
data["winner_age"] = data["winner_age"].fillna(
    data["winner_age"].median()
)
data["loser_age"] = data["loser_age"].fillna(
    data["loser_age"].median()
)

print("Cleaned shape:", data.shape)
print(data[required_cols].isnull().sum())

```

3. IMPORTANT: Preserve these columns

Do not remove: winner_id, loser_id, winner_rank, loser_rank, winner_rank_points, loser_rank_points, winner_age, loser_age, surface, year and tourney_date (if available). The IDs and chronological fields are required to calculate historical features without future-data leakage.

Do not use post-match statistics as model inputs: aces, double faults, first-serve percentage, break points, winners, etc. These are match outcomes/statistics and are not available before the match.

4. Create chronological historical features

This step creates features using only information available BEFORE each match. For every match, first calculate the players' previous overall win rate, surface win rate and head-to-head record; only after creating those features should the current match be added to the player's history.

```

history_data = data.copy()

# Sort chronologically
history_data = history_data.sort_values(
    by=["year", "tourney_date"]
).reset_index(drop=True)

player_stats = {}
h2h = {}
historical_rows = []

def get_player_stats(player_id):
    if player_id not in player_stats:
        player_stats[player_id] = {
            "matches": 0,
            "wins": 0,
            "surface_matches": {},
            "surface_wins": {}
        }
    return player_stats[player_id]

def get_h2h(player1, player2):
    key = tuple(sorted([player1, player2]))
    if key not in h2h:
        h2h[key] = {player1: 0, player2: 0}
    return h2h[key].get(player1, 0)

for _, row in history_data.iterrows():
    winner = row["winner_id"]
    loser = row["loser_id"]
    surface = row["surface"]

    if pd.isna(winner) or pd.isna(loser):
        continue

    w = get_player_stats(winner)
    l = get_player_stats(loser)

    w_win_rate = w["wins"] / w["matches"] if w["matches"] else 0.5
    l_win_rate = l["wins"] / l["matches"] if l["matches"] else 0.5

    w_sm = w["surface_matches"].get(surface, 0)
    w_sw = w["surface_wins"].get(surface, 0)
    l_sm = l["surface_matches"].get(surface, 0)
    l_sw = l["surface_wins"].get(surface, 0)

```

```

w_surface_rate = w_sw / w_sm if w_sm else 0.5
l_surface_rate = l_sw / l_sm if l_sm else 0.5

w_h2h = get_h2h(winner, loser)
l_h2h = get_h2h(loser, winner)

# Winner perspective: target = 1
historical_rows.append({
    "player1": winner,
    "player2": loser,
    "surface": surface,
    "rank1": row["winner_rank"],
    "rank2": row["loser_rank"],
    "points1": row["winner_rank_points"],
    "points2": row["loser_rank_points"],
    "age1": row["winner_age"],
    "age2": row["loser_age"],
    "winrate1": w_win_rate,
    "winrate2": l_win_rate,
    "surfacerate1": w_surface_rate,
    "surfacerate2": l_surface_rate,
    "h2h1": w_h2h,
    "h2h2": l_h2h,
    "target": 1,
    "year": row["year"]
})

# Loser perspective: target = 0
historical_rows.append({
    "player1": loser,
    "player2": winner,
    "surface": surface,
    "rank1": row["loser_rank"],
    "rank2": row["winner_rank"],
    "points1": row["loser_rank_points"],
    "points2": row["winner_rank_points"],
    "age1": row["loser_age"],
    "age2": row["winner_age"],
    "winrate1": l_win_rate,
    "winrate2": w_win_rate,
    "surfacerate1": l_surface_rate,
    "surfacerate2": w_surface_rate,
    "h2h1": l_h2h,
    "h2h2": w_h2h,
    "target": 0,
    "year": row["year"]
})

# Update history AFTER this match
w["matches"] += 1
w["wins"] += 1
l["matches"] += 1

w["surface_matches"][surface] = w["surface_matches"].get(surface, 0) + 1
w["surface_wins"][surface] = w["surface_wins"].get(surface, 0) + 1
l["surface_matches"][surface] = l["surface_matches"].get(surface, 0) + 1

key = tuple(sorted([winner, loser]))
if key not in h2h:
    h2h[key] = {winner: 0, loser: 0}
h2h[key][winner] = h2h[key].get(winner, 0) + 1

historical_data = pd.DataFrame(historical_rows)

print("Historical dataset:", historical_data.shape)
display(historical_data.head())

```

5. Convert historical information into final ML features

```
improved_data = pd.DataFrame()

improved_data["rank_diff"] = (
    historical_data["rank1"] -
    historical_data["rank2"]
)

improved_data["rank_points_diff"] = (
    historical_data["points1"] -
    historical_data["points2"]
)

improved_data["age_diff"] = (
    historical_data["age1"] -
    historical_data["age2"]
)

improved_data["win_rate_diff"] = (
    historical_data["winrate1"] -
    historical_data["winrate2"]
)

improved_data["surface_rate_diff"] = (
    historical_data["surfacerate1"] -
    historical_data["surfacerate2"]
)

improved_data["h2h_diff"] = (
    historical_data["h2h1"] -
    historical_data["h2h2"]
)

improved_data["surface"] = historical_data["surface"]
improved_data["year"] = historical_data["year"]
improved_data["target"] = historical_data["target"]

numeric_features = [
    "rank_diff",
    "rank_points_diff",
    "age_diff",
    "win_rate_diff",
    "surface_rate_diff",
    "h2h_diff"
]

for col in numeric_features:
    improved_data[col] = improved_data[col].fillna(
        improved_data[col].median()
    )

print("Final feature dataset:", improved_data.shape)
display(improved_data.head())
```

6. Chronological train/test split — FINAL HANDOFF

```
# 2018-2023 = training
# 2024 = testing

train_data = improved_data[
    improved_data["year"] < 2024
].copy()

test_data = improved_data[
    improved_data["year"] == 2024
].copy()

print("Training samples:", len(train_data))
print("Testing samples:", len(test_data))

print("Training years:",
      sorted(train_data["year"].unique()))

print("Testing years:",
      sorted(test_data["year"].unique()))

# Save both files for the model-training teammate
train_data.to_csv(
    "train_data.csv",
    index=False
)

test_data.to_csv(
    "test_data.csv",
    index=False
)
```

```
print("Saved:")
print(" - train_data.csv")
print(" - test_data.csv")
```

7. What the teammate should push to GitHub

```
data/
  train_data.csv
  test_data.csv

notebooks/
  data_cleaning_and_feature_engineering.ipynb

Optional:
  cleaned_atp_data.csv

Commit message:
"Add data preprocessing, historical features and train-test split"

Then send me:
- GitHub branch/commit
- train_data.csv
- test_data.csv
- notebook
- final train/test row counts
```

8. Your starting point

Once these files are pushed, you start from model training. You do NOT need to repeat the raw-data cleaning or historical feature engineering. Load `train_data.csv` and `test_data.csv`, separate X/y , preprocess the categorical surface feature, and train Logistic Regression, Random Forest and SVM.